Data Structure Lab 1: Matrix Multiplication and Strassen's algorithm

叶锦灏 24300750085

Summary

In the lab, I implemented the ordinary matrix multiplication and Strassen's algorithm based on a self-designed matrix class, and test their running time to verify their time complexity. It shows that Strassen's algorithm runs much faster than the ordinary algorithm in large matrixs.

Key Code Design

Matrix Class

In order to do the experiment conveniently and efficiently, we need to design a good data structure to implement the matrix as a mathematical object. Our data structure needs to hit these requirements:

1. We need to store the data of a matrix in a basic data structure, rather than complex structures like `std::vector<std::vector<T>>`, which brings more time and

memory consumption with their redundant design and unnecessary inner operations.

2. We need to implement high-efficient sub-block reference, since Strassen's algorithm includes lots of sub-blocking operations, rather than keep copying sub-matrixs which takes more consumption.

3. We need to make the access, modification and add/sub operations simple and elegant interfaces, hiding their complex inner design and ugly memory indexing.

To hit all of these requirements above, I firstly designed a matrix class before implementing the two multiplication algorithms. Here shows the basic structure of the matrix class:

```
template<typename T>
class Matrix{
    private:
        std::shared_ptr<T[]> data_; // smart prt to continous mem
        size_t rows_;
        size_t cols_;
        size_t ld_; // how many unit do two rows in the matrix seperate, for pa
        size_t offset_;

    public:
        Matrix(): data_(nullptr), rows_(0), cols_(0), ld_(0), offset_(0){}

        Matrix(size_t rows, size_t cols)
            : data_(new T[rows*cols](), std::default_delete<T[]>()),
            rows_(rows), cols_(cols), ld_(cols), offset_(0){}

        Matrix(std::shared_ptr<T[]> data,
               size_t rows, size_t cols, size_t ld, size_t offset)
            : data_(std::move(data)),rows_(rows),cols_(cols),ld_(ld),offset_(of

    //  ... omitted the following code
}
```

In the class, a matrix is stored on a continous memory area be `new` ed. In order to make the matrix blocking easier, I come up with two core parameters: `ld_` and `offset_`. `ld_` means leading distance, describing how far apart the two rows of the matrix in the

memory. `offset_` describes where the first element is. By implementing these two attribute, we can create sub block as creating "view windows": we use a shared pointer to point to its memory area, and by using the four parameters can we calculate out how the rows of the sub matrix actually distributed seperately in the memory area. Here provides the core matrix blocking algorithm code:

```
Matrix SubMatrix(size_t i0, size_t j0, size_t h, size_t w) const {
    assert(i0 + h ≤ rows_ && j0 + w ≤ cols_);
    return Matrix(data_, h, w, ld_, offset_ + i0 * ld_ + j0);
}
```

We can access the elements of submatrix by the operation form `(i,j)` as same as we are accessing the original matrix. And, no more memory spaces are applied during the blocking operations, we only create a new window to pretend a totally unique and indenpent matrix.

What's more, we overloaded the `+` `-` `(i,j)` operators to let the matrix operations make more sense. The detailed code of the matrix class, see in `matrix.h`.

Matrix Multiplication

I implemented the ordinary and Strassen's matrix multiplication algorithm. Here shows the key code of the Strassen's algorithm:

```
template<typename T>
Matrix<T> matrixMulStrassen_padded(const Matrix<T>& A, const Matrix<T>& B){
    assert(A.cols() == B.rows());
    assert(A.cols() == A.rows() && B.cols() == B.rows());
    // Strassen算法要求A和B都是n = 2^k的方阵，所以要做一个padding。这个函数是默认已经
    size_t n = A.cols();
    if(n≤32){
        return matrixMul(A, B);
    }

    auto [A11, A12, A21, A22] = matrixSplit4(A);
    auto [B11, B12, B21, B22] = matrixSplit4(B);
```

```
    Matrix<T> M1, M2, M3, M4, M5, M6, M7;
    M1 = matrixMulStrassen_padded(A11 + A22, B11 + B22);
    M2 = matrixMulStrassen_padded(A21 + A22, B11);
    M3 = matrixMulStrassen_padded(A11, B12 - B22);
    M4 = matrixMulStrassen_padded(A22, B21 - B11);
    M5 = matrixMulStrassen_padded(A11 + A12, B22);
    M6 = matrixMulStrassen_padded(A21 - A11, B11 + B12);
    M7 = matrixMulStrassen_padded(A12 - A22, B21 + B22);

    Matrix<T> C11, C12, C21, C22;
    C11 = M1 + M4 - M5 + M7;
    C12 = M3 + M5;
    C21 = M2 + M4;
    C22 = M1 - M2 + M3 + M6;

    return matrix4CombineTo1(C11, C12, C21, C22, n/2);
}

template <typename T>
Matrix<T> matrixMulStrassen(const Matrix<T>& A, const Matrix<T>& B){
    assert(A.cols()==B.rows());
    const size_t m = A.rows();
    const size_t n = B.cols();

    auto [A_pad, B_pad] = matrixPadding(A, B);
    Matrix<T> C_pad = matrixMulStrassen_padded(A_pad, B_pad);

    // Unpadding 回原始 m×n
    Matrix<T> C(m, n);
    for (size_t i = 0; i < m; ++i) {
        std::memcpy(C.data() + i * C.ld(),
                    C_pad.data() + i * C_pad.ld(),
                    n * sizeof(T));
    }
    return C;
}
```

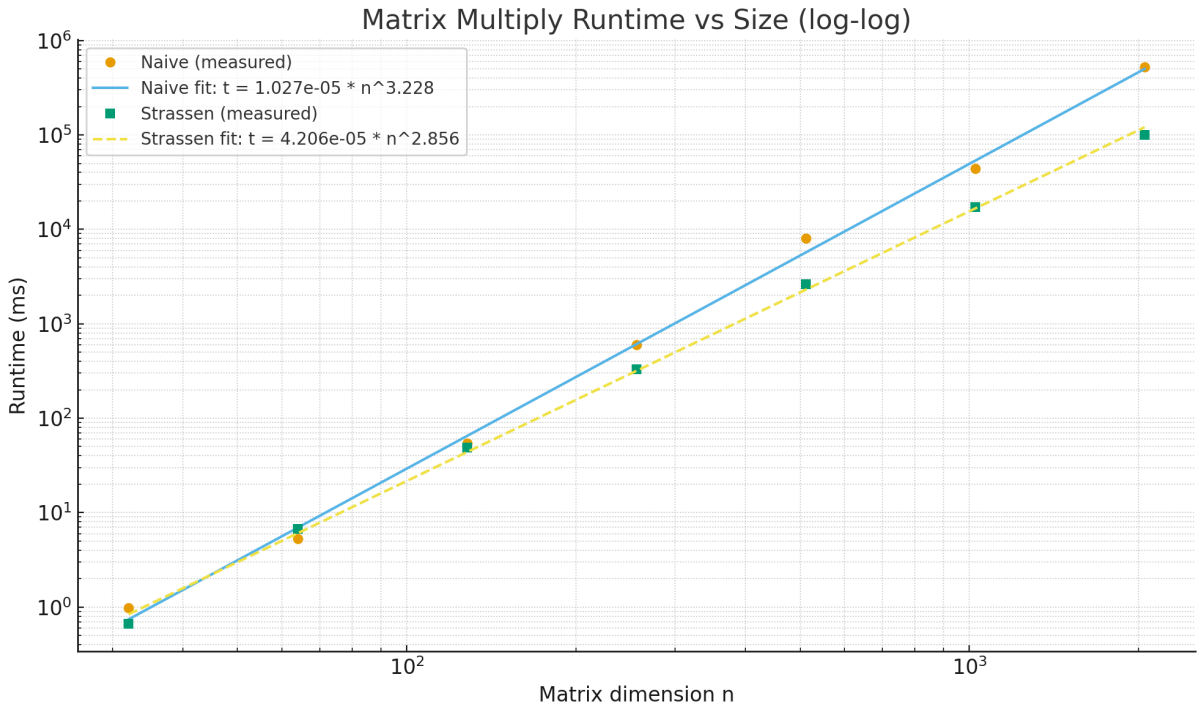
Two points worth noticing. First, the Strassen's algorithm requires square matrix whose scale is $n \times n$, and $n = 2^k$. So, if the input matrix is not square, or its side length is not the form of 2^k, it would be padded to the form of 2^k. And second, if the recursion creates

`n ≤ 32` matrixs, it would directly calls the ordinary matrix multiplication, which is faster in small scaled matrixs.

More detailed code see in `matrix_multiplication.h`

Experiment

I tested the running time of both algorithms on matrixs of different scales, from `32*32` to `2048*2048`, each case multiplies 2 than it's former one. Here the graph shows the result in ogarithmic coordinate system, with the fitted line in the form of $y = cn^{\alpha}$.



We can see the tested time complexity fitted below:

Algorithm	Theoretical Time Exponent	Tested Time Exponent
Ordinary Algorithm	3	3.228
Strassen's Algorithm	$\log_2 7 = 2.81$	2.856

The tested time exponent is almost fitted to the theoretical value(The regression $R^2 > 0.998$, while the bias rate is 7.6% for the ordinary and 1.6% for the Strassen's algorithm.

Error Analysis

There are two primary sources of error in this experiment:

1. **Random Initialization:**
   Since all matrices were filled with uniformly random values, minor floating-point rounding differences accumulate during recursion.
   However, as shown by the maximum absolute difference $< 10^{-12}$, the numerical stability remains acceptable.

2. **System-Level Noise:**
   The timing results are subject to fluctuations caused by CPU scheduling, cache warm-up, and background processes.
   Multiple runs (averaged or taking the best case) were performed to mitigate this effect.

Therefore, the overall deviation between the measured and theoretical exponents can be attributed mainly to constant-factor overheads (memory allocation, copy, and recursion depth), rather than algorithmic errors.